



Chapter 7: A Closer Look at Methods and Classes

Object Oriented Programming Language (JAVA)

Md. Eamin Rahman



Method Overloading

- **Definition:** In Java, a class can define two or more methods that share the **same name**, provided their **parameter declarations are different**.
- **The Signature Rule:** Java uses the **method signature** (name + parameter types) and the **arguments provided** to determine which version of the method to call.
- **Polymorphism:** Method overloading is one of the primary ways Java supports **compile-time polymorphism**.

Method Signature

- A method's **signature** consists of its **name** and its **parameter list** (types and order)
- The return type is **not** part of the **signature**

Component	Part of Signature?
Method name	 Yes
Parameter types & order	 Yes
Return type	 No
Parameter names	 No

Rules for Method Overloading

- To overload methods, they must have the **same name**
- Overloaded methods must differ in their **parameter list**:
 - **Number** of parameters
 - **Type** of parameters
 - **Order** of parameter types

Invalid Overloading

Changing only the **return type** does **not** create a new overloaded method.

```
class Test {  
    int square(int x) {  
        return x * x;  
    }  
    // Error: differs only by return type  
    double square(int x) {  
        return x * x;  
    }  
}
```

- The **parameter list** must change.
- The **return type alone** is not sufficient to distinguish methods.

Overloading Example

```
1 class OverloadDemo {  
2     void test() {  
3         IO.println("No parameters");  
4     }  
5     void test(int a) {  
6         IO.println("a: " + a);  
7     }  
8     void test(int a, int b) {  
9         IO.println("a and b: " + a + " " + b);  
10    }  
11    double test(double a) {  
12        IO.println("double a: " + a);  
13        return a*a;  
14    }  
15 }
```

Overloading Example

```
1 class Overload {  
2     public static void main(String[] args) {  
3         OverloadDemo ob = new OverloadDemo();  
4         double result;  
5         ob.test();  
6         ob.test(10);  
7         ob.test(10, 20);  
8         result = ob.test(123.25);  
9         IO.println("Result of ob.test(123.25): " + result);  
10    }  
11 }
```

Overload Resolution & Type Conversion

- When an overloaded method is called, Java finds a **match** between **call arguments** and **method parameters**
- The match need not always be **exact** — Java's **automatic type conversions** can apply
- If no exact match exists, Java **promotes** the argument to a compatible type and resolves accordingly

⚠ Automatic conversion can cause **ambiguity errors** if multiple overloaded versions become equally valid

Overloading and Type Conversion

```
1 class OverloadDemo {
2     void test() {
3         IO.println("No parameters");
4     }
5     void test(int a, int b) {
6         IO.println("a and b: " + a + " " + b);
7     }
8     void test(double a) {
9         IO.println("Inside test(double) a: " + a);
10    }
11 }
12 class Overload {
13     public static void main(String[] args) {
14         OverloadDemo ob = new OverloadDemo();
15         ob.test();
16         ob.test(10, 20);
17         ob.test(88); // this will invoke test(double)
18         ob.test(123.2); // this will invoke test(double)
19     }
20 }
```

Overloading and Type Conversion

```
1 class OverloadDemo {  
2     void test() {  
3         IO.println("No parameters");  
4     }  
5     void test(int a, int b) {  
6         IO.println("a and b: " + a + " " + b);  
7     }  
8     void test(double a) {  
9         IO.println("Inside test(double) a: " + a);  
10    }  
11 }  
12 class Overload {  
13     public static void main(String[] args) {  
14         OverloadDemo ob = new OverloadDemo();  
15         ob.test();  
16         ob.test(10, 20);  
17         ob.test(88); // this will invoke test(double)  
18         ob.test(123.2); // this will invoke test(double)  
19     }  
20 }
```

Console Output

```
No parameters  
a and b: 10 20  
Inside test(double) a: 88.0  
Inside test(double) a: 123.2
```

Overloading and Ambiguity

```
1 class OverloadDemo {
2     void test(int a, double b) {
3         IO.println("Inside test(int, double)");
4     }
5     void test(double a, int b) {
6         IO.println("Inside test(double, int)");
7     }
8 }
9 class Overload {
10     public static void main(String[] args) {
11         OverloadDemo ob = new OverloadDemo();
12         ob.test(10, 20); // Error: ambiguous call
13     }
14 }
```

⚠ This code will not compile due to an **ambiguity error**. The call `ob.test(10, 20)` could match either `test(int a, double b)` or `test(double a, int b)` after type conversion, and Java cannot determine which method to call.

Overloading Constructors

- **Constructors** can also be overloaded in Java, just like regular methods.
- This allows a class to initialize objects in multiple ways.

```
1  class Box {  
2      double width, height, depth;  
3      // Constructor with three parameters  
4      Box(double w, double h, double d) {  
5          width = w;  
6          height = h;  
7          depth = d;  
8      }  
9      // Method to calculate volume  
10     double volume() {  
11         return width * height * depth;  
12     }  
13 }
```

Overloading Constructors

```
1  class Box {  
2      double width, height, depth;  
3      // Constructor with three parameters  
4      Box(double w, double h, double d) {  
5          width = w;  
6          height = h;  
7          depth = d;  
8      }  
9      // Constructor with one parameter (cube)  
10     Box(double len) {  
11         width = height = depth = len;  
12     }  
13     // Default constructor  
14     Box() {  
15         width = height = depth = 0;  
16     }  
17     // Method to calculate volume  
18     double volume() {  
19         return width * height * depth;  
20     }  
21 }
```

Overloading Constructors

```
1 class OverloadCons {
2     public static void main(String[] args) {
3         // create boxes using the various constructors
4         Box mybox1 = new Box(10, 20, 15);
5         Box mybox2 = new Box();
6         Box mycube = new Box(7);
7         double vol;
8         // get volume of first box
9         vol = mybox1.volume();
10        IO.println("Volume of mybox1 is " + vol);
11        // get volume of second box
12        vol = mybox2.volume();
13        IO.println("Volume of mybox2 is " + vol);
14        // get volume of cube
15        vol = mycube.volume();
16        IO.println("Volume of mycube is " + vol);
17    }
18 }
```

Overloading Constructors

```
1 class OverloadCons {
2     public static void main(String[] args) {
3         // create boxes using the various constructors
4         Box mybox1 = new Box(10, 20, 15);
5         Box mybox2 = new Box();
6         Box mycube = new Box(7);
7         double vol;
8         // get volume of first box
9         vol = mybox1.volume();
10        IO.println("Volume of mybox1 is " + vol);
11        // get volume of second box
12        vol = mybox2.volume();
13        IO.println("Volume of mybox2 is " + vol);
14        // get volume of cube
15        vol = mycube.volume();
16        IO.println("Volume of mycube is " + vol);
17    }
18 }
```

Console Output

Volume of mybox1 is 3000.0
Volume of mybox2 is 0.0
Volume of mycube is 343.0

Using Objects as Parameters

```
1  class Test {
2      int a, b;
3      Test(int i, int j) {
4          a = i;
5          b = j;
6      }
7      boolean equalTo(Test o) {
8          if(o.a == a && o.b == b) return true;
9          else return false;
10     }
11 }
12 class PassOb {
13     public static void main(String[] args) {
14         Test ob1 = new Test(100, 22);
15         Test ob2 = new Test(100, 22);
16         Test ob3 = new Test(-1, -1);
17         IO.println("ob1 == ob2: " + ob1.equalTo(ob2));
18         IO.println("ob1 == ob3: " + ob1.equalTo(ob3));
19     }
20 }
```

Using Objects as Parameters

```
1 class Test {
2     int a, b;
3     Test(int i, int j) {
4         a = i;
5         b = j;
6     }
7     boolean equalTo(Test o) {
8         if(o.a == a && o.b == b) return true;
9         else return false;
10    }
11 }
12 class PassOb {
13     public static void main(String[] args) {
14         Test ob1 = new Test(100, 22);
15         Test ob2 = new Test(100, 22);
16         Test ob3 = new Test(-1, -1);
17         IO.println("ob1 == ob2: " + ob1.equalTo(ob2));
18         IO.println("ob1 == ob3: " + ob1.equalTo(ob3));
19     }
20 }
```

Console Output

```
ob1 == ob2: true
ob1 == ob3: false
```

Cloning The **Box** Object

```
1  class Box {
2      double width, height, depth;
3      Box(double w, double h, double d) {
4          width = w; height = h; depth = d;
5      }
6      // Notice this constructor. It takes an object of type Box.
7      Box(Box ob) { // pass object to constructor
8          width = ob.width; height = ob.height; depth = ob.depth;
9      }
10     Box(double len) {
11         width = height = depth = len;
12     }
13     Box() {
14         width = height = depth = 0;
15     }
16     double volume() {
17         return width * height * depth;
18     }
19 }
```

Cloning The **Box** Object

```
1 class OverloadCons2 {
2     public static void main(String[] args) {
3         // create boxes using the various constructors
4         Box mybox1 = new Box(10, 20, 15);
5         Box mybox2 = new Box();
6         Box mycube = new Box(7);
7         Box myclone = new Box(mybox1); // create copy of mybox1
8         // prints volume of first box
9         IO.println("Volume of mybox1 is " + mybox1.volume());
10        // prints volume of second box
11        IO.println("Volume of mybox2 is " + mybox2.volume());
12        // prints volume of cube
13        IO.println("Volume of cube is " + mycube.volume());
14        // prints volume of clone
15        IO.println("Volume of clone is " + myclone.volume());
16    }
17 }
```

Cloning The **Box** Object

```
1 class OverloadCons2 {
2     public static void main(String[] args) {
3         // create boxes using the various constructors
4         Box mybox1 = new Box(10, 20, 15);
5         Box mybox2 = new Box();
6         Box mycube = new Box(7);
7         Box myclone = new Box(mybox1); // create copy of mybox1
8         // prints volume of first box
9         IO.println("Volume of mybox1 is " + mybox1.volume());
10        // prints volume of second box
11        IO.println("Volume of mybox2 is " + mybox2.volume());
12        // prints volume of cube
13        IO.println("Volume of cube is " + mycube.volume());
14        // prints volume of clone
15        IO.println("Volume of clone is " + myclone.volume());
16    }
17 }
```

Console Output

```
Volume of mybox1 is 3000.0
Volume of mybox2 is 0.0
Volume of cube is 343.0
Volume of clone is 3000.0
```

Argument Passing: Two Approaches

Call-by-Value

- A **copy** of the argument is passed.
- Changes to the parameter do **not affect** the original argument.

Call-by-Reference

- A **reference** to the original argument is passed.
- Changes through the parameter **affect** the original argument.

 Java uses **call-by-value** for all arguments — but the effect differs by type

Java and Call-by-Value

The effect depends on what is being passed:

Argument Type	What is Copied?	Original Value Affected?
Primitive type	Actual value	✗ No
Object reference	Reference value	✓ Object state can change

⚠ Java never passes arguments by true call-by-reference.

Passing Primitive Types

- Method receives a **copy** of the primitive value.
- Modifying the parameter **affects** only the **local copy**.
- The original variable remains **unchanged** outside the method.

Passing Primitive Types

```
1 class Test {
2     void meth(int i, int j) {
3         i *= 2;
4         j /= 2;
5     }
6 }
7 class CallByValue {
8     public static void main(String[] args) {
9         Test ob = new Test();
10        int a = 15, b = 20;
11        IO.println("a and b before call: " + a + " " + b);
12        ob.meth(a, b);
13        IO.println("a and b after call: " + a + " " + b);
14    }
15 }
```

Passing Primitive Types

```
1 class Test {  
2     void meth(int i, int j) {  
3         i *= 2;  
4         j /= 2;  
5     }  
6 }  
7 class CallByValue {  
8     public static void main(String[] args) {  
9         Test ob = new Test();  
10        int a = 15, b = 20;  
11        IO.println("a and b before call: " + a + " " + b);  
12        ob.meth(a, b);  
13        IO.println("a and b after call: " + a + " " + b);  
14    }  
15 }
```

Console Output

a and b before call: 15 20
a and b after call: 15 20

Passing Objects (Reference Types)

- A **copy of the reference** is passed.
- Both references point to the **same object**.
- Changes to the object's fields **do affect** the original

Passing Objects (Reference Types)

```
1 class Test {
2     int a, b;
3     Test(int i, int j) {
4         a = i;
5         b = j;
6     }
7     void meth(Test o) { // pass an object
8         o.a *= 2;
9         o.b /= 2;
10    }
11 }
12 class PassObjRef {
13     public static void main(String[] args) {
14         Test ob = new Test(15, 20);
15         IO.println("ob.a and ob.b before call: " + ob.a + " " + ob.b);
16         ob.meth(ob);
17         IO.println("ob.a and ob.b after call: " + ob.a + " " + ob.b);
18     }
19 }
```

Passing Objects (Reference Types)

```
1 class Test {
2     int a, b;
3     Test(int i, int j) {
4         a = i;
5         b = j;
6     }
7     void meth(Test o) { // pass an object
8         o.a *= 2;
9         o.b /= 2;
10    }
11 }
12 class PassObjRef {
13     public static void main(String[] args) {
14         Test ob = new Test(15, 20);
15         IO.println("ob.a and ob.b before call: " + ob.a + " " + ob.b);
16         ob.meth(ob);
17         IO.println("ob.a and ob.b after call: " + ob.a + " " + ob.b);
18     }
19 }
```

Console Output

ob.a and ob.b before call: 15 20
ob.a and ob.b after call: 30 10

Returning Objects from Methods

- Methods can **return objects**, just like any other type
- Each call can create and return **new objects**
- The original object remains **unchanged** unless explicitly modified
- Returned objects can be assigned to **reference variables** and used normally

Returning Objects from Methods

```
1  class Test {
2      int a;
3      Test(int i) {
4          a = i;
5      }
6      Test incrByTen() {
7          Test temp = new Test(a+10);
8          return temp;
9      }
10 }
11 class RetOb {
12     public static void main(String[] args) {
13         Test ob1 = new Test(2);
14         Test ob2;
15         ob2 = ob1.incrByTen();
16         IO.println("ob1.a: " + ob1.a);
17         IO.println("ob2.a: " + ob2.a);
18         ob2 = ob2.incrByTen();
19         IO.println("ob2.a after second increase: " + ob2.a);
20     }
21 }
```

Returning Objects from Methods

```
1 class Test {  
2     int a;  
3     Test(int i) {  
4         a = i;  
5     }  
6     Test incrByTen() {  
7         Test temp = new Test(a+10);  
8         return temp;  
9     }  
10 }  
11 class RetOb {  
12     public static void main(String[] args) {  
13         Test ob1 = new Test(2);  
14         Test ob2;  
15         ob2 = ob1.incrByTen();  
16         IO.println("ob1.a: " + ob1.a);  
17         IO.println("ob2.a: " + ob2.a);  
18         ob2 = ob2.incrByTen();  
19         IO.println("ob2.a after second increase: " + ob2.a);  
20     }  
21 }
```

Console Output

```
ob1.a: 2  
ob2.a: 12  
ob2.a after second increase: 22
```

Recursion

- **Recursion** — defining something in terms of itself
- A method that **calls itself** is called a **recursive method**
- Every recursive method needs a **base case** to stop the recursion

```
int fact(int n) {  
    if (n == 1) return 1;           // base case  
    return fact(n - 1) * n;        // recursive call  
}
```

- `fact(3)` → `fact(2)` → `fact(1)` → returns `1`
- Unwinds: `1 × 2 × 3 = 6`

Access Control and Encapsulation

- **Encapsulation** combines:
 - **Data** (fields)
 - **Methods** that operate on that data
- A key benefit of encapsulation is **access control**.
- Access control helps:
 - Prevent misuse of class members
 - Protect internal data
 - Maintain object integrity

Access Modifiers in Java

Java provides the following access levels:

Modifier	Accessible From
<code>public</code>	Anywhere
<code>private</code>	Only within the same class
<code>protected</code>	Class, package, and subclasses*
<i>(default)</i>	Same package only



`protected` is mainly important when **inheritance** is used.

public Access

- A `public` member can be accessed from **any code** that can access the class.

```
public int count;
```

Example

```
obj.count = 10;    // Allowed
```

private Access

- A `private` member can be accessed **only within its own class**.
- Hidden from all external code.

```
private int balance;
```

Example

```
obj.balance = 100; // Error
```



`private` is commonly used to protect sensitive data.

Default (Package-Private) Access

When no access modifier is specified:

```
int value;
```

- Accessible within the **same package**.
- Not accessible from outside the package.



This is called **default** or **package-private** access.

Why Is `main()` Public?

The Java runtime system calls `main()` from outside your class.

```
public static void main(String[] args)
```

Therefore, `main()` must be declared as:

- `public`
- So the Java runtime can access it.

Example: Different Access Levels

```
1 class Test {
2     int a;           // default access
3     public int b;    // public access
4     private int c;   // private access
5 }
6 class AccessTest {
7     public static void main(String[] args) {
8         Test ob = new Test();
9         // These are OK, a and b may be accessed directly
10        ob.a = 10;
11        ob.b = 20;
12        IO.println("a, b: " + ob.a + " " + ob.b);
13        // This is not OK and will cause an error
14        // ob.c = 100; // Error!
15        // IO.println("c: " + ob.c); // Error!
16    }
17 }
```

Accessing Private Data Through Methods

```
1  class Test {
2      private int c;
3
4      void setc(int i) {
5          c = i;
6      }
7
8      int getc() {
9          return c;
10     }
11 }
12 class AccessTest {
13     public static void main(String[] args) {
14         Test ob = new Test();
15         ob.setc(100); // OK
16         IO.println(ob.getc()); // OK
17     }
18 }
```

Why Use Getter and Setter Methods?

⇒ Direct Access 

```
ob.c = 100; // Not allowed
```

⇒ Controlled Access 

```
ob.setc(100); // Allowed
```



Benefits:

- Validate data before storing
- Prevent accidental modification
- Hide implementation details

Practical Example: Stack Class

Internal Data

```
class Stack {  
    private int[] stck = new int[10];  
    private int tos;  
}
```

Both members are private:

- **stck** → stores stack elements
- **tos** → index of the top element

Controlled Stack Operations

The stack can only be manipulated through methods.

```
void push(int item) { ... }  
  
int pop() { ... }
```

Users interact with the stack through:

- `push()`
- `pop()`

instead of modifying internal variables directly.

Why Make Stack Data Private?

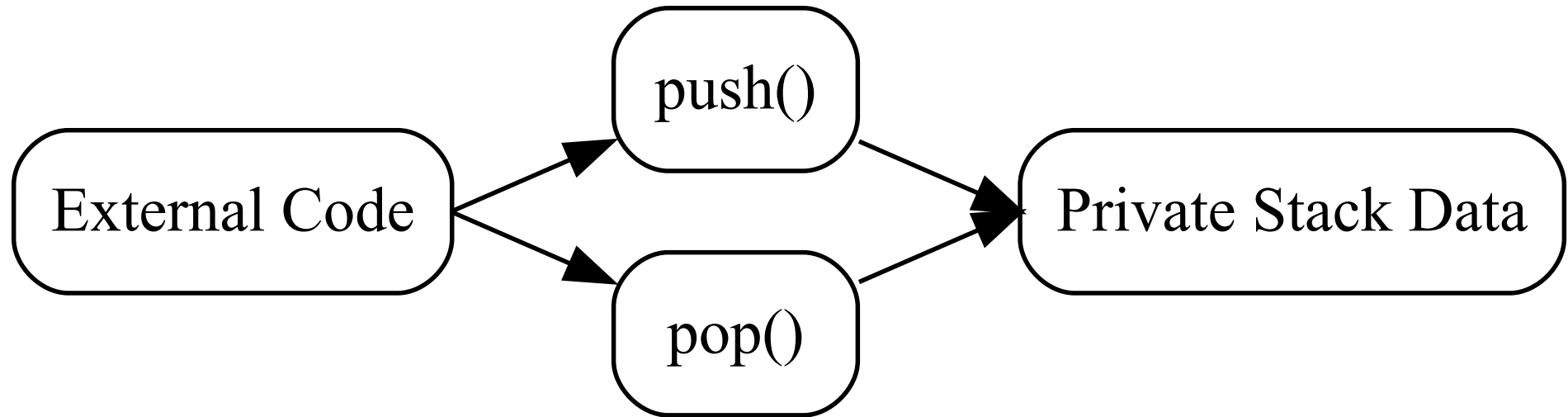
Without **private**, external code could do:

```
stack.tos = 100;
```

This could:

- **Corrupt** stack state
- Cause **invalid** array access
- **Break** program logic

Encapsulation Protects the Object



- Internal details remain **hidden**.
- Object **integrity** is preserved.

The **static** Keyword

Normally, class members belong to **objects (instances)**.

A member declared with **static** belongs to the **class itself**, not to any object.

```
class Demo {  
    static int count;  
}
```

Key Characteristics

- Exists **once per class**
- Shared by **all objects**
- Can be accessed **without creating an object**

Instance Members vs Static Members

Instance Member	Static Member
Belongs to an object	Belongs to the class
Each object has its own copy	Only one shared copy exists
Access using an object	Access using the class name



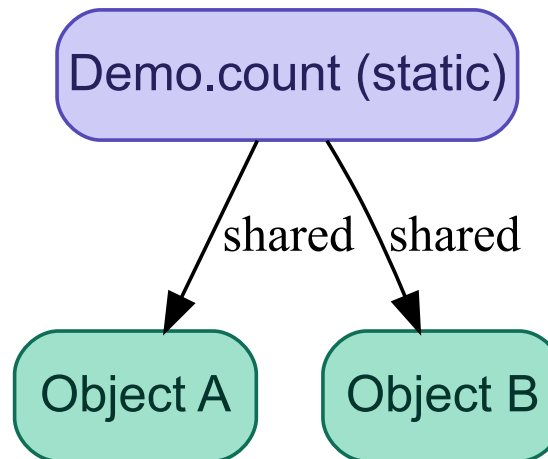
Static members exist before any objects are created and can be accessed without creating an object.

Static Variables

A **static** variable is shared by all objects of the class.

```
class Demo {  
    static int count = 0;  
}
```

Shared Among All Objects



No separate copy is created for each object.

Static Methods

A **static** method belongs to the class.

```
class Demo {  
    static void show() {  
        IO.println("Hello");  
    }  
}
```

Called without creating an object:

```
Demo.show();
```

Why Is `main()` Static?

```
public static void main(String[] args)
```

`main()` is `static` because:

- The Java runtime calls it **before any object exists**.
- No instance of the class is required to start program execution.

Restrictions of Static Methods

A `static` method:

-  Can directly access **static variables**
-  Can directly call **other static methods**
-  Cannot directly access instance variables
-  Cannot directly call instance methods
-  Cannot use `this` or `super`



Static methods operate without any specific object.

Accessing Static Members

Outside the class, access static members using:

```
ClassName.member
```

Example:

```
class StaticDemo {  
    static int b = 99;  
  
    static void callme() {  
        IO.println("Hello");  
    }  
}  
  
StaticDemo.callme();  
IO.println(StaticDemo.b);
```



Using the *class* name is the preferred way to access *static* members.

Blocks in Java

Java supports several types of blocks:

- Method blocks
- Control blocks
 - `if`
 - `for`
 - `while`
 - `switch`
- Static initialization blocks
- Instance initialization blocks

Static Initialization Block

A **static block** initializes static data.

```
class UseStatic {  
    static int a = 3;  
    static int b;  
  
    static {  
        b = a * 4;  
    }  
}
```

Characteristics

- Executes **once**
- Runs when the class is **first loaded**
- Useful for complex static initialization

Example: Static Block Execution

```
1  class UseStatic {  
2      static int a = 3;  
3      static int b;  
4  
5      static {  
6          IO.println("Static block initialized.");  
7          b = a * 4;  
8      }  
9  
10     static void meth(int x) {  
11         IO.println("a = " + a);  
12         IO.println("b = " + b);  
13     }  
14  
15     public static void main(String[] args) {  
16         meth(42);  
17     }  
18 }
```

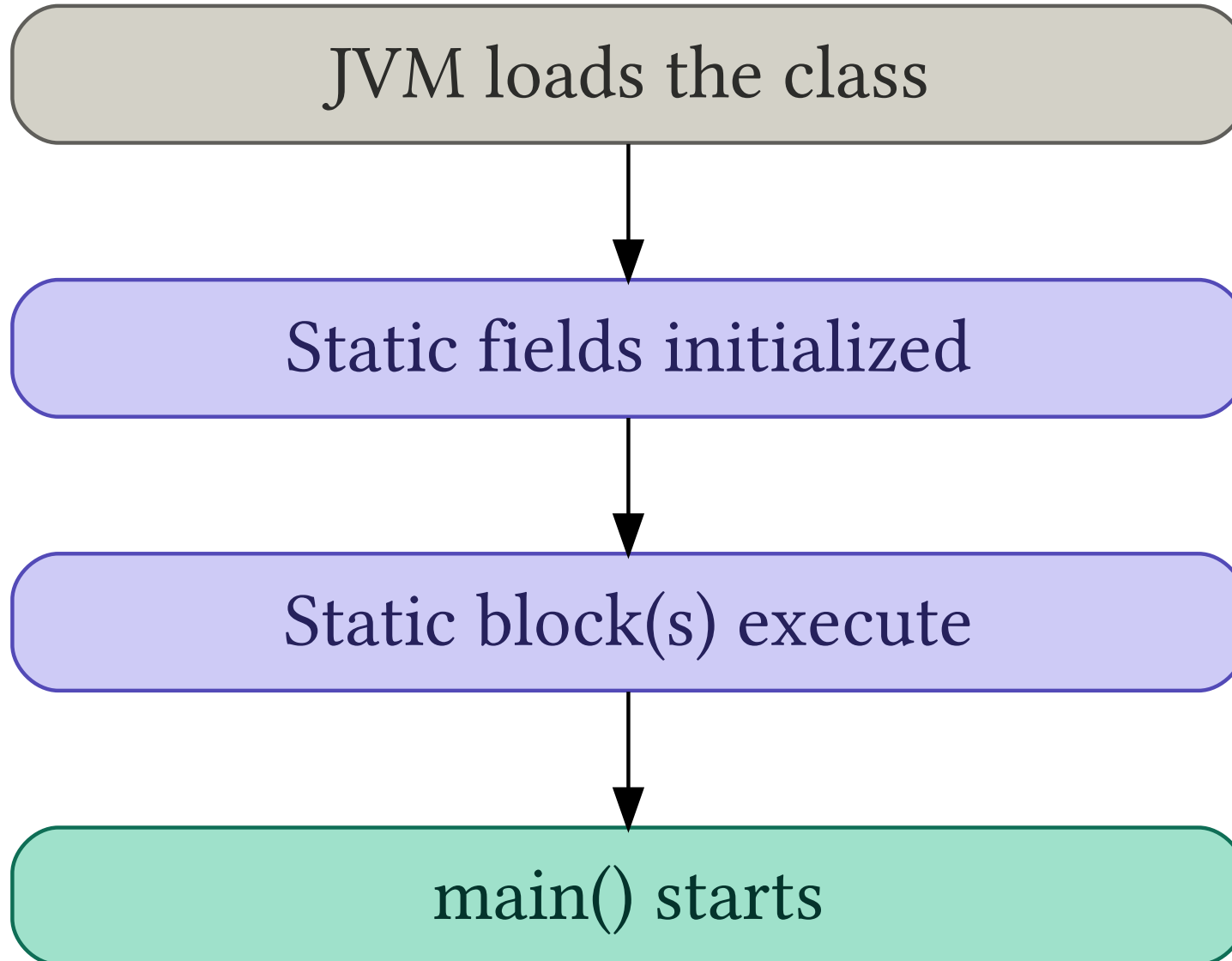
Example: Static Block Execution

```
1  class UseStatic {  
2      static int a = 3;  
3      static int b;  
4  
5      static {  
6          IO.println("Static block initialized.");  
7          b = a * 4;  
8      }  
9  
10     static void meth(int x) {  
11         IO.println("a = " + a);  
12         IO.println("b = " + b);  
13     }  
14  
15     public static void main(String[] args) {  
16         meth(42);  
17     }  
18 }
```

Console Output

```
Static block initialized.  
a = 3  
b = 12
```

Execution Order



Instance Initialization Block

An **instance initialization block** is a block of code inside a class that:

- Is not inside any method
- Is not marked **static**
- Runs automatically whenever an object is created

```
class Demo {  
    {  
        IO.println("Instance block");  
    }  
    Demo() {  
        IO.println("Constructor");  
    }  
}
```

Why Use an Instance Block?

Useful when the same initialization code is needed by **multiple constructors**.

Instead of repeating code:

```
Constructor A
|
|— Initialization Code
|
Constructor B
|— Initialization Code
```

Place the common code in an instance block.

Example

```
1  class Demo {  
2      int x;  
3  
4      {  
5          x = 100;  
6          IO.println("Instance block");  
7      }  
8  
9      Demo() {  
10         IO.println("Default constructor");  
11     }  
12  
13     Demo(int n) {  
14         x = n;  
15         IO.println("Parameterized constructor");  
16     }  
17 }
```

Example

```
1  class Demo {  
2      int x;  
3  
4      {  
5          x = 100;  
6          IO.println("Instance block");  
7      }  
8  
9      Demo() {  
10         IO.println("Default constructor");  
11     }  
12  
13     Demo(int n) {  
14         x = n;  
15         IO.println("Parameterized constructor");  
16     }  
17 }
```

Console Output

```
Demo obj1 = new Demo();
```

```
Instance block  
Default constructor
```

Example

```
1  class Demo {  
2      int x;  
3  
4      {  
5          x = 100;  
6          IO.println("Instance block");  
7      }  
8  
9      Demo() {  
10         IO.println("Default constructor");  
11     }  
12  
13     Demo(int n) {  
14         x = n;  
15         IO.println("Parameterized constructor");  
16     }  
17 }
```

Console Output

```
Demo obj1 = new Demo();  
Instance block  
Default constructor
```

Console Output

```
Demo obj1 = new Demo(5);  
Instance block  
Parameterized constructor
```

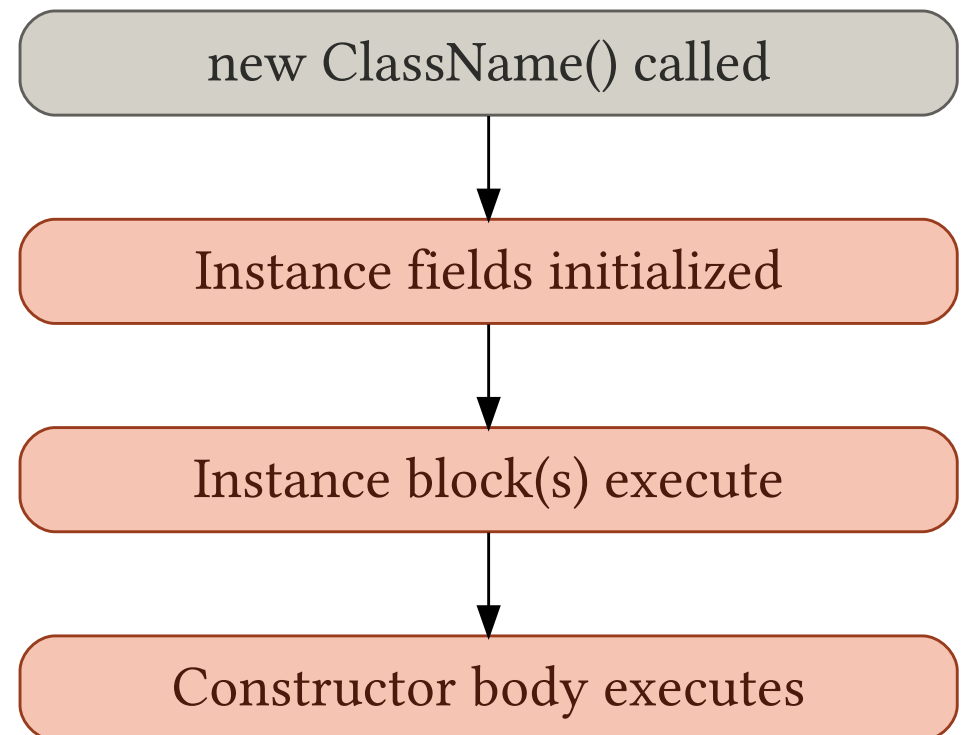
Execution Order

Creating an object:

```
Demo obj = new Demo();
```

- The instance block always executes **before** the constructor.
- The instance block runs **every time** an object is created, regardless of which constructor is called.

Execution sequence:



Multiple Instance Blocks

A class may contain multiple instance blocks.

```
class Demo {  
    {  
        IO.println("Block 1");  
    }  
    {  
        IO.println("Block 2");  
    }  
    Demo() {  
        IO.println("Constructor");  
    }  
}
```

Multiple Instance Blocks

A class may contain multiple instance blocks.

```
class Demo {  
    {  
        IO.println("Block 1");  
    }  
    {  
        IO.println("Block 2");  
    }  
    Demo() {  
        IO.println("Constructor");  
    }  
}
```

Console Output

```
Block 1  
Block 2  
Constructor
```

Multiple Instance Blocks

A class may contain multiple instance blocks.

```
class Demo {  
    {  
        IO.println("Block 1");  
    }  
    {  
        IO.println("Block 2");  
    }  
    Demo() {  
        IO.println("Constructor");  
    }  
}
```

Console Output

```
Block 1  
Block 2  
Constructor
```



They execute in the order they appear.

Instance Block vs Static Block

Instance Block	Static Block
No <code>static</code> keyword	Declared with <code>static</code>
Runs for every object	Runs once per class
Executes before each constructor	Executes when the class is first loaded
Can access instance and static members	Can directly access only static members

Limitations of Instance Blocks

- Cannot accept parameters.
- Cannot be called explicitly.
- Executes **only during object creation**.
- Usually less common than constructors.



When To Use an Instance Block?

✓ Good for:

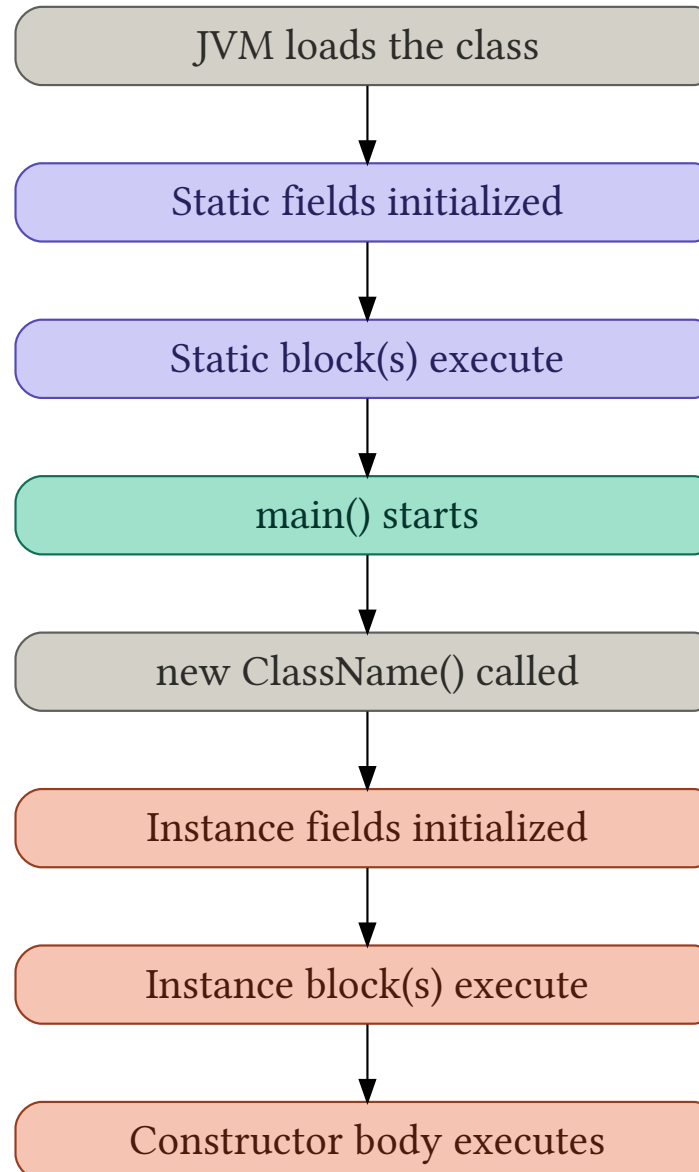
- Common initialization shared by multiple constructors
- Reducing duplicate constructor code

✗ Avoid when:

- Initialization belongs to only one constructor
- Constructor code is already simple and clear

💡 In most Java programs, **constructors** are preferred, and instance blocks are used only for *specialized* initialization.

Execution Order



The *final* Keyword

The *final* keyword is used to **prevent modification**.

It can be applied to:

- **Variables (fields)**
- **Method parameters**
- **Local variables**
- **Methods (covered later)**
- **Classes (covered later)**

final Fields (Constants)

A **final** field can be assigned **only once**.

```
final int MAX_SIZE = 100;
```

- Must be initialized **before it is used**.
- After initialization, its value **cannot be changed**.

```
MAX_SIZE = 200;    // Error
```

💡 A **final** field behaves like a **constant**.

Naming Convention for Constants

By convention, *final* constants are written in **UPPERCASE**.

```
final int FILE_NEW    = 1;  
final int FILE_OPEN  = 2;  
final int FILE_SAVE   = 3;  
final int FILE_SAVEAS = 4;  
final int FILE_QUIT   = 5;
```

This improves readability and distinguishes constants from regular variables.

final Parameters

A method parameter can also be declared *final*.

```
void print(final int x) {  
    IO.println(x);  
}
```

Attempting to modify it causes a compile-time error.

```
x++;           // Error
```

Benefit

- Prevents accidental modification of method arguments.

final Local Variables

A local variable declared *final* can be assigned **only once**.

```
final int age = 20;
```

Reassigning it is not allowed.

```
age = 25;    // Error
```

Benefit

- Makes code safer by preventing unintended reassignment.

Arrays Revisited

In Java, **arrays are objects**.

Because arrays are objects, they:

- Are created using **new**
- Store their size in a built-in field named **length**
- Are accessed through a reference variable

```
int[] nums = new int[5];
```

The **length** Field

Every array has a built-in **instance variable**:

```
arrayName.length
```

It returns the **size (capacity)** of the array.

```
int[] nums = new int[5];  
  
IO.println(nums.length);    // 5
```



length is a **field**, not a method.

length vs Number of Used Elements

length represents the **total capacity** of the array.

It **does not** indicate how many elements currently contain meaningful values.

```
int[] nums = new int[5];
```

```
nums[0] = 10;
```

```
nums[1] = 20;
```

Property	Value
nums.length	5
Elements assigned	2

💡 The array can hold 5 elements, even though only 2 have been assigned.

Nested Classes

A **nested class** is a class defined **inside another class**.

```
class Outer {  
  
    class Inner {  
        // Nested class  
    }  
}
```

Key Characteristics

- Exists only within the **outer class**.
- Its **scope** is limited to the enclosing class.
- Cannot exist **independently** of the outer class.

Benefits of Nested Classes

Nested classes help:

- Group related classes together
- Improve code organization
- **Hide** implementation details
- Increase **encapsulation**

Types of Nested Classes

Java supports **two main types** of nested classes:

Type	Declared With	Requires Outer Object?
Static Nested Class	<code>static</code>	 No
Inner Class	(no <code>static</code>)	 Yes

 A nested class declared inside a method or block is called a **local class**.

Static Nested Class

A static nested class belongs to the **outer class**, not to an object of the outer class.

Characteristics

- Declared using `static`
- Can be created without an object of the outer class
- Can directly access **only static members** of the outer class

Example: Static Nested Class

```
class Outer {  
    static class Nested {  
        void show() {  
            IO.println("Static nested class");  
        }  
    }  
}
```

Creating and using the static nested class:

```
Outer.Nested nested = new Outer.Nested();  
nested.show();
```

Output:

```
Static nested class
```

Inner Class (Non-static Nested Class)

A non-static nested class is called an **inner class**.

Characteristics

- Belongs to an **object** of the outer class
- Can directly access from the outer class:
 - Instance variables & methods
 - Static variables & methods
 - Even **private** members

Example: Inner Class

```
class Outer {  
    int outerX = 100;  
  
    class Inner {  
        void display() {  
            IO.println(outerX);  
        }  
    }  
}
```

Creating and using the inner class:

```
Outer outer = new Outer();  
Outer.Inner inner = outer.new Inner();  
  
inner.display();
```

Output:

```
100
```

Access: Inner Class → Outer Class

✓ Allowed

```
class Outer {  
    int outerX = 100;  
    class Inner {  
        void display() {  
            IO.println(outerX);  
        }  
    }  
}
```

Access: Outer Class → Inner Class

✗ Not allowed directly

```
class Outer {  
    class Inner {  
        int y = 10;  
    }  
  
    void showY() {  
        IO.println(y);    // Error  
    }  
}
```

The outer class must first create an **Inner** object.

Local Classes

A nested class can also be declared **inside a method or block**.

Characteristics

- Visible only inside that method/block
- Cannot be accessed from outside
- Useful for helper classes with limited scope

Example: Local Class

```
void test() {  
  
    class Inner {  
        void display() {  
            IO.println("Local class");  
        }  
    }  
  
    Inner obj = new Inner();  
    obj.display();  
}
```

The **String** Class

In Java, every string is an object of the **String** class.

```
String msg = "Hello";
```

Even a string literal is a **String** object.

```
IO.println("This is a String");
```

String Objects Are Immutable

A `String` object is **immutable**.



Immutable means its contents cannot be changed after creation.

```
String s = "Java";
```

The following does **not** modify the original string:

```
s = s + " Programming";
```

Instead:

1. A new `String` object is created.
2. `s` now refers to the new object.
3. The old object remains **unchanged**.

Why Is Immutability Useful?

Several benefits of immutable strings:

- Safer and more reliable programs
- Strings can be shared efficiently
- Easier to use as keys in collections (e.g., `HashMap`)

If frequent modification is required, Java provides:

- `StringBuilder`
- `StringBuffer`

These classes allow string contents to be changed.

String Concatenation

The **+** operator joins (concatenates) strings.

```
String first = "Java";  
String second = "Programming";  
  
String result = first + " " + second;
```

Output:

```
Java Programming
```

Common **String** Methods

Method	Purpose
<code>length()</code>	Returns the number of characters
<code>charAt(i)</code>	Returns the character at index <code>i</code>
<code>equals()</code>	Compares the contents of two strings

Common **String** Methods

Method	Purpose
<code>toUpperCase()</code>	Converts to uppercase
<code>toLowerCase()</code>	Converts to lowercase
<code>substring(start, end)</code>	Extracts a substring
<code>replace(oldChar, newChar)</code>	Replaces characters

Example: `length()` and `charAt()`

```
String str = "First String";  
  
IO.println(str.length());  
IO.println(str.charAt(3));
```

Output:

```
12  
S
```



String indexing starts at 0.

Comparing Strings

To compare string contents, use `equals()`.

```
String s1 = "Java";  
String s2 = "Java";  
  
IO.println(s1.equals(s2));
```

Output:

```
true
```



`equals()` compares the **contents** of strings.

Example Program

```
String strOb1 = "First String";  
String strOb2 = "Second String";  
String strOb3 = strOb1;  
  
IO.println(strOb1.length());  
IO.println(strOb1.charAt(3));  
  
IO.println(strOb1.equals(strOb2));  
IO.println(strOb1.equals(strOb3));
```

Output:

```
12  
s  
false  
true
```

Example: String `substring()` and `replace()`

```
String str = "Hello World";  
IO.println(str.substring(0, 5));  
IO.println(str.replace('o', 'a'));  
IO.println(str.replaceAll("ll", "LL"));
```

Output:

```
Hello  
Hella World  
HeLLo World
```

Command-Line Arguments

Sometimes a program needs **input** when it starts.

This input is provided using **command-line arguments**.

```
java Program Alice 20
```

Here:

- **Program** → class name
- **Alice** → first argument
- **20** → second argument

`main()` Receives the Arguments

The `main()` method receives all command-line arguments in the `args` array.

```
public static void main(String[] args)
```

- `args` is an array of `String`.
- Every command-line argument is stored as a **string**.

Accessing Arguments

Arguments are accessed using **array indexing**.

Argument	Array Element
First	<code>args[0]</code>
Second	<code>args[1]</code>
Third	<code>args[2]</code>
...	...

Example:

```
java Program Alice 20 CSE
```

Results in:

```
args[0] → "Alice"  
args[1] → "20"  
args[2] → "CSE"
```

Example Program

```
class CommandLineDemo {  
    public static void main(String[] args) {  
  
        IO.println("Name: " + args[0]);  
        IO.println("Age: " + args[1]);  
    }  
}
```

Run

```
java CommandLineDemo Alice 20
```

Output

```
Name: Alice  
Age: 20
```

Processing Multiple Arguments

The `length` field tells how many arguments were supplied.

```
for (int i = 0; i < args.length; i++) {  
    IO.println(args[i]);  
}
```

This prints every command-line argument.

Important Notes

- All command-line arguments are stored as **String** objects.
- Numeric values must be converted before performing arithmetic.

```
int age = Integer.parseInt(args[0]);
```

Varargs (Variable-Length Arguments)

Varargs allow a method to accept a **variable number of arguments**.

- **Varargs = Variable-Length Arguments**
- Also called a **variable-arity method**.
- Eliminates the need to write multiple overloaded methods.

Declaring a Varargs Method

A varargs parameter is declared using `...`.

```
static void vaTest(int... v) {  
    // ...  
}
```

This means the method can be called with:

- Zero arguments
- One argument
- Many arguments

How Varargs Work

Inside the method, the varargs parameter acts like an array.

```
static void vaTest(int... v) {  
    IO.println(v.length);  
    for (int x : v) IO.print(x + " ");  
}
```

The compiler automatically creates an `int[]` array.

Calling a Varargs Method

```
vaTest(10); // 1 argument
```

Output:

```
Number of args: 1  
10
```

```
vaTest(1,2,3); // 3 arguments
```

Output:

```
Number of args: 3  
1 2 3
```

```
vaTest(); // 0 arguments
```

Output:

```
Number of args: 0
```

Complete Example

```
class VarArgs {  
    static void vaTest(int... v) {  
        IO.print("Number of args: " + v.length + " Contents: ");  
  
        for (int x : v) IO.print(x + " ");  
        IO.println();  
    }  
  
    public static void main(String[] args) {  
        vaTest(10);  
        vaTest(1, 2, 3);  
        vaTest();  
    }  
}
```

Output:

```
Number of args: 1 Contents: 10  
Number of args: 3 Contents: 1 2 3  
Number of args: 0 Contents:
```

Varargs with Other Parameters

A method may have **regular parameters** together with a **varargs parameter**.

```
int doIt(int a, int b, double c, int... vals)
```

Here:

- **a**, **b**, and **c** are normal parameters.
- **vals** is the varargs parameter.

Rules for Varargs

Rule 1: Varargs Must Be Last

 Valid

```
void test(int x, String... names)
```

 Invalid

```
void test(int... nums, boolean flag)
```

The varargs parameter **must be the last parameter.**

Rules for Varargs (Continued)

Rule 2: Only One Varargs Parameter

 Valid

```
void test(int... nums)
```

 Invalid

```
void test(int... nums, double... values)
```

A method can have **only one** varargs parameter.

Why Use Varargs?

Without varargs, you may need multiple overloaded methods:

```
sum()  
sum(int a)  
sum(int a, int b)  
sum(int a, int b, int c)  
...
```

With varargs:

```
sum(int... values)
```

One method handles **any number of arguments**, making code **simpler** and **more flexible**.

Overloading Varargs Methods

A **varargs method** can be overloaded just like a normal method.

Java chooses the correct version based on the:

- **Number of arguments**
- **Types of arguments**

Method 1: Different Vararg Types

Varargs methods can be overloaded by changing the **type** of the varargs parameter.

```
static void vaTest(int... v) { }  
  
static void vaTest(boolean... v) { }
```

Example Calls

```
vaTest(1, 2, 3);  
  
vaTest(true, false, true);
```

Java selects the appropriate method based on the **argument type**.

Method 2: Add Normal Parameters

You can also overload a varargs method by adding **regular parameters**.

```
static void vaTest(String msg, int... v) { }
```

Example

```
vaTest("Testing:", 10, 20);
```

Java uses both:

- The **number** of arguments
- The **types** of arguments

to determine which overloaded method to call.

Example

```
class VarArgs3 {  
  
    static void vaTest(int... v) { }  
  
    static void vaTest(boolean... v) { }  
  
    static void vaTest(String msg, int... v) { }  
  
    public static void main(String[] args) {  
  
        vaTest(1, 2, 3);  
  
        vaTest("Testing:", 10, 20);  
  
        vaTest(true, false, false);  
  
    }  
}
```

How Java Chooses the Method

Method	Example Call
<code>vaTest(<i>int</i>...)</code>	<code>vaTest(1, 2, 3)</code>
<code>vaTest(<i>boolean</i>...)</code>	<code>vaTest(true, false)</code>
<code>vaTest(<i>String</i>, <i>int</i>...)</code>	<code>vaTest("Testing:", 10, 20)</code>

The compiler selects the **best matching overloaded method**.

Varargs and Ambiguity

Sometimes **overloading varargs methods** can confuse the compiler.

If **more than one overloaded method** is an equally good match, the call becomes **ambiguous**, resulting in a **compile-time error**.

Example 1: Ambiguous Overload

```
static void vaTest(int... v) { }  
  
static void vaTest(int n, int... v) { }
```

Now consider:

```
vaTest(1);
```

Which Method Should Java Call?

Option 1:

```
vaTest(int... v)
```

- `v = {1}`

Option 2:

```
vaTest(int n, int... v)
```

- `n = 1` and `v = { }` (empty)

! Both methods are valid matches, so the compiler reports an **ambiguity error**.

Why Does This Happen?

A varargs parameter can contain **zero or more arguments**.

```
int... v
```

means ↴

```
0, 1, 2, 3, ...
```

↳ arguments are all valid.

Because the varargs array may be **empty**, Java cannot always determine which overloaded method is intended.

Example 2: Empty Varargs

```
static void vaTest(int... v) { }
```

```
static void vaTest(boolean... v) { }
```



Valid calls:

```
vaTest(1, 2, 3);
```

```
vaTest(true, false);
```



But what about?

```
vaTest();
```

Why Is `vaTest()` Ambiguous?

The call ↴

```
vaTest();
```

⇒ matches **both** methods:

```
vaTest(int... v)
```

```
vaTest(boolean... v)
```

with an empty `int[]`

with an empty `boolean[]`

⚠ Since both are equally valid, the **compiler cannot decide**.

Result:

```
Compile-time Error: Reference to vaTest is ambiguous
```

Example Program

```
class VarArgs4 {  
  
    static void vaTest(int... v) { }  
  
    static void vaTest(boolean... v) { }  
  
    public static void main(String[] args) {  
  
        vaTest(1, 2, 3);           // OK  
  
        vaTest(true, false);      // OK  
  
        vaTest();                  // Compile-time error  
    }  
}
```

Avoiding Ambiguity

When overloading varargs methods:

- Avoid overloads that can match the **same method call**.
- Ensure each method has a **clearly distinguishable parameter list**.
- Be especially careful when **zero arguments** are allowed.

Local Variable Type Inference (**var**)

Java supports **local variable type inference** using the **var** keyword.

Instead of explicitly writing the type, the compiler **infers** it from the initializer.

```
var myStr = "This is a string";
```

The compiler infers:

```
String myStr = "This is a string";
```

 **var** does **not** mean the variable has no type—the type is determined at compile time.

Using **var** with Reference Types

var is especially useful for **reference types**, whose class names may be long.

Traditional Declaration

```
FileInputStream fin = new FileInputStream("test.txt");
```

Using **var**

```
var fin = new FileInputStream("test.txt");
```

Both declarations create the **same type** of variable.

Example

```
class MyClass {  
    private int i;  
    MyClass(int k) { i = k; }  
  
    int geti() { return i; }  
  
    void seti(int k) { if (k >= 0) i = k; }  
}
```

Using **var**:

```
var mc = new MyClass(10);  
IO.println(mc.geti());  
mc.seti(19);
```

The compiler infers:

```
MyClass mc = new MyClass(10);
```

Rules for Using `var`

`var` can only be used for local variables.

The variable **must be initialized**.

 Valid

```
var count = 10;  
  
var name = "Alice";
```

 Invalid

```
var x;           // Error
```

The compiler cannot infer the type without an initializer.

Where Can `var` Be Used?

Location	Allowed?
Local variables	 Yes
Enhanced <code>for</code> loop variables	 Yes
Traditional <code>for</code> loop variables	 Yes
Method parameters	 No
Return types	 No
Instance variables (fields)	 No
Static variables	 No

Benefits of `var`

- Reduces repetitive type declarations.
- Improves readability for long class names.
- Makes code cleaner without changing type safety.

💡 The inferred type is **fixed at compile time**; Java remains a **statically typed** language.

